

Projeto Full Stack

(Parte 3)

Tela de Login Segura no Frontend (Next.js)

Objetivo

Criar uma tela de login funcional no **Next.js**, integrada ao backend NestJS, com boas práticas de segurança como:

- Armazenamento seguro de senhas (hash no backend, nunca no frontend).

- Uso de **JWT (JSON Web Token)** para autenticação.

- Proteção de rotas no frontend.

- Validação de entradas do usuário.

1. Preparação do Backend (NestJS)

Antes de criar o frontend, precisamos garantir que o backend suporte login seguro.

1.1 Instalar dependências

No backend:

```
npm install bcryptjs @nestjs/jwt passport-jwt
```

1.2 Criar módulo de autenticação

Gerar módulo de autenticação:

```
nest generate module auth
nest generate service auth
nest generate controller auth
```

1.3 Implementar hash de senha

No cadastro de usuários, nunca salvar senha em texto puro:

```
import * as bcrypt from 'bcryptjs';

async createUser(dto: CreateUserDto) {
  const salt = await bcrypt.genSalt(10);
  const hashedPassword = await bcrypt.hash(dto.password, salt);
  const user = { ...dto, password: hashedPassword };
  return this.userRepo.save(user);
}
```

1.4 Gerar token JWT no login

```
import { JwtService } from '@nestjs/jwt';

async login(dto: LoginDto) {
```

```

const user = await this.userRepo.findOneBy({ CPF_CNPJ: dto.CPF_CNPJ });
if (!user || !(await bcrypt.compare(dto.password, user.password))) {
  throw new Error("Credenciais inválidas");
}
const payload = { sub: user.ID_PESSOA, nome: user.NOME };
return { access_token: this.jwtService.sign(payload) };
}

```

2. Preparação do Frontend (Next.js)

2.1 Instalar dependências

```
npm install axios js-cookie
```

2.2 Criar serviço de autenticação

Arquivo src/services/auth.ts:

```

import api from "../api";
import Cookies from "js-cookie";

export async function login(cpf_cnpj: string, password: string) {
  const res = await api.post("/auth/login", { cpf_cnpj, password });
  Cookies.set("token", res.data.access_token, { secure: true });
  return res.data;
}

export function logout() {
  Cookies.remove("token");
}

export function getToken() {
  return Cookies.get("token");
}

```

3. Tela de Login

Arquivo src/app/login/page.tsx:

typescript

```

"use client";
import { useState } from "react";
import { login } from "../../services/auth";
import { useRouter } from "next/navigation";

export default function LoginPage() {
  const [cpf_cnpj, setCpfCnpj] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const router = useRouter();

  const handleSubmit = async (e: any) => {
    e.preventDefault();
    try {
      await login(cpf_cnpj, password);
      router.push("/dashboard");
    } catch {

```

```

        setError("Credenciais inválidas");
    }
};

return (
  <div className="flex flex-col items-center justify-center h-screen">
    <h1 className="text-2xl font-bold mb-4">Login</h1>
    <form onSubmit={handleSubmit} className="flex flex-col gap-2 w-64">
      <input
        type="text"
        placeholder="CPF/CNPJ"
        value={cpf_cnpj}
        onChange={(e) => setCpfCnpj(e.target.value)}
        className="border p-2"
      />
      <input
        type="password"
        placeholder="Senha"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        className="border p-2"
      />
      {error && <p className="text-red-500">{error}</p>}
      <button className="bg-blue-500 text-white p-2">Entrar</button>
    </form>
  </div>
);
}

```

4. Proteção de Rotas

Criar **middleware** para verificar se o usuário está logado antes de acessar páginas restritas.

Arquivo `src/middleware.ts`:

```

import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import Cookies from "js-cookie";

export function middleware(req: NextRequest) {
  const token = req.cookies.get("token");
  if (!token && req.nextUrl.pathname.startsWith("/dashboard")) {
    return NextResponse.redirect(new URL("/login", req.url));
  }
}

```

5. Boas Práticas de Segurança

Nunca armazenar senha no frontend. Apenas enviar para o backend.

Sempre usar HTTPS em produção.

Usar JWT com expiração curta (ex: 15 minutos) e refresh tokens.

Evitar mensagens de erro detalhadas (não dizer “usuário não existe”, apenas “credenciais inválidas”).

Resultado até aqui:

Tela de login funcional no frontend.

Usuário só acessa o **dashboard** se estiver autenticado.

Senhas protegidas com hash no backend.

Token JWT armazenado em cookie seguro.

Integração da Tela de Login com o Sistema

1. Fluxo de Autenticação

- O usuário acessa `/login`.
- Ao enviar CPF/CNPJ e senha, o backend valida e retorna um **JWT**.
- O frontend armazena esse token em **cookies seguros**.
- Todas as requisições subsequentes (CRUD de Pessoas, Moradores, Reservas, etc.) enviam o token no **header Authorization**.
- Se o token não existir ou for inválido, o backend retorna erro 401 (não autorizado).

2. Ajustando o Serviço `api.ts`

Adicionar interceptador para incluir o token automaticamente em cada requisição:

```
import axios from "axios";
import Cookies from "js-cookie";

const api = axios.create({
  baseURL: "http://localhost:3000",
});

// Interceptador para adicionar token
api.interceptors.request.use((config) => {
  const token = Cookies.get("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

==> Assim, qualquer chamada `api.get("/pessoas")` ou `api.post("/moradores")` já enviará o token.

3. Protegendo o Dashboard

No arquivo `src/app/dashboard/page.tsx`, verificar se o usuário está autenticado:

```

"use client";
import { useEffect } from "react";
import { useRouter } from "next/navigation";
import Cookies from "js-cookie";

export default function Dashboard() {
  const router = useRouter();

  useEffect(() => {
    const token = Cookies.get("token");
    if (!token) {
      router.push("/login");
    }
  }, []);

  return (
    <div className="p-6">
      <h1 className="text-2xl font-bold">Painel do Condomínio</h1>
      <div className="grid grid-cols-2 gap-4 mt-6">
        <a href="/pessoas" className="border p-4">Pessoas</a>
        <a href="/moradores" className="border p-4">Moradores</a>
        <a href="/funcionarios" className="border p-4">Funcionários</a>
        <a href="/fornecedores" className="border p-4">Fornecedores</a>
        <a href="/visitantes" className="border p-4">Visitantes</a>
        <a href="/unidades" className="border p-4">Unidades</a>
        <a href="/areas-comuns" className="border p-4">Áreas Comuns</a>
        <a href="/reservas" className="border p-4">Reservas</a>
        <a href="/boletos" className="border p-4">Boletos</a>
        <a href="/comunicados" className="border p-4">Comunicados</a>
        <a href="/contratos" className="border p-4">Contratos</a>
        <a href="/financeiro" className="border p-4">Financeiro</a>
      </div>
    </div>
  );
}

```

4. Integração com CRUDs Existentes

Agora que o login está implementado, todas as rotas CRUD já criadas no frontend deverão passar pelo interceptador do Axios, garantindo que o token seja enviado. Se o token expirar ou for inválido, o backend retorna erro 401 → o frontend pode redirecionar para /login.

- **Pessoas**
- **Moradores**
- **Reservas, Boletos, Comunicados, etc**

Exemplo de tratamento no CRUD de Pessoas:

```

try {
  const res = await api.get("/pessoas");
  setPessoas(res.data);
} catch (err: any) {
  if (err.response?.status === 401) {
    router.push("/login");
  }
}

```

Resultados

Usuário precisa **fazer login** para acessar o dashboard e qualquer CRUD.

Todas as requisições ao backend são autenticadas com **JWT**.

Se não houver login, o sistema redireciona para `/login`.

Integração completa entre **login seguro** e as funcionalidades já implementadas (Pessoas, Moradores, Reservas, Boletos, etc.).